Ejemplo implementación de colonias de hormigas

https://github.com/ppoffice/ant-colony-tsp

Inteligencia Artificial

Universidad del Magdalena

Esquema de representación

```
class _Ant(object):
    def __init__(self, aco: ACO, graph: Graph):
        self.colony = aco
        self.graph = graph
        self.total_cost = 0.0
        self.tabu = []  # tabu list
        # the local increase of pheromone
        self.pheromone_delta = []
        # nodes which are allowed for the next selection
        self.allowed = [i for i in range(graph.rank)]
        self.eta = [[0 if i == j else 1 / graph.matrix[i][j]
                     for j in range(graph.rank)] for i in range(graph.rank)]
        start = random.randint(0, graph.rank - 1)  # start from any node
        self.tabu.append(start)
        self.current = start
        self.allowed.remove(start)
```

Función objetivo

Distancia Euclidiana:

$$D(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

```
def distance(city1: dict, city2: dict):
    return math.sqrt((city1['x'] - city2['x']) ** 2 +
                     (city1['y'] - city2['y']) ** 2)
```

```
def main():
    cities = []
    points = []
    with open('./data/chn45.txt') as f:
        for line in f.readlines():
            line = [float(x.replace("\n", "")) for x in line.split(" ")]
            cities.append(dict(index=int(line[0]), x=float(line[1]), y=float(line[2])))
            points.append((line[1], line[2]))
    cost_matrix = []

    rank = len(cities)
    for i in range(rank):
        row = []
        for j in range(rank):
            row.append(distance(cities[i], cities[j]))
        cost_matrix.append(row)
```

Algoritmo

Begin

Inicialización

Mientras no se satisfaga condición de parada :

Se posiciona cada hormiga en un nodo de inicio

Repetir:

Para cada hormiga :

Escoger el próximo nodo

Actualizar feromona paso por paso

Fin del para

Hasta que cada hormiga construya una solución

Aplicar la actualización de las feromona

Actualizar la mejor solución

Fin del mientras

Fin del algoritmos

Función principal en Python

```python
def solve(self, graph: Graph):
    """
    :param graph:
    """
    best_cost = float('inf')
    best_solution = []
    for gen in range(self.generations):
        # noinspection PyUnusedLocal
        ants = [_Ant(self, graph)
                for i in range(self.ant_count)]
        for ant in ants:
            for i in range(graph.rank - 1):
                ant._select_next()
            ant.total_cost +=
                graph.matrix[ant.tabu[-1]][ant.tabu[0]]
            if ant.total_cost < best_cost:
                best_cost = ant.total_cost
                best_solution = [] + ant.tabu
            # update pheromone
            ant._update_pheromone_delta()
        self._update_pheromone(graph, ants)
    return best_solution, best_cost
```

Colonias de Hormiga

- Regla de transición:

$$p_{ij}^{k}(t) = \begin{cases} \dfrac{[\tau_{ij}(t)]^{\alpha}[\eta_{ij}]^{\beta}}{\sum_{k \in allowed_k} [\tau_{ik}(t)]^{\alpha}[\eta_{ik}]^{\beta}} & \text{if } j \in \text{allowed}_k \\ 0 & \text{otherwise} \end{cases}$$

- Actualización de la feromona

$$\tau_{ij}(t+1) = \rho\tau_{ij}(t) + \Delta\tau_{ij}$$

$$\Delta\tau_{i,j}^{k} = \begin{cases} \dfrac{Q}{L_k} & if\,(i,j) \in \text{bestTour} \\ 0 & \text{otherwise} \end{cases}$$

ρ Tasa de decremento de la feromona

Función de transición

$$p_{ij}^{k}(t) = \begin{cases} \dfrac{[\tau_{ij}(t)]^{\alpha}[\eta_{ij}]^{\beta}}{\sum_{k \in allowed_k} [\tau_{ik}(t)]^{\alpha}[\eta_{ik}]^{\beta}} & \text{if } j \in \text{allowed}_k \\ 0 & \text{otherwise} \end{cases}$$

```
def _select_next(self):
    denominator = 0
    for i in self.allowed:
        denominator += self.graph.pheromone[self.current][i] ** self.colony.alpha *
                       self.eta[self.current][i] ** self.colony.beta
    # noinspection PyUnusedLocal
    #probabilities for moving to a node in the next step
    probabilities = [0 for i in range(self.graph.rank)]
    for i in range(self.graph.rank):
        try:
            self.allowed.index(i)  # test if allowed list contains i
            probabilities[i] = self.graph.pheromone[self.current][i] ** self.colony.alpha * \
                self.eta[self.current][i] ** self.colony.beta / denominator
        except ValueError:
            pass  # do nothing
    # select next node by probability roulette
    selected = 0
    rand = random.random()
    for i, probability in enumerate(probabilities):
        rand -= probability
        if rand <= 0:
            selected = i
            break
    self.allowed.remove(selected)
    self.tabu.append(selected)
    self.total_cost += self.graph.matrix[self.current][selected]
    self.current = selected
```

$$\Delta\tau_{i,j}^{k} = \begin{cases} \dfrac{Q}{L_k} & if\ (i,j) \in \text{bestTour} \\ 0 & \text{otherwise} \end{cases}$$

```python
# noinspection PyUnusedLocal
def _update_pheromone_delta(self):
    self.pheromone_delta = [[0 for j in range(self.graph.rank)]
                            for i in range(self.graph.rank)]
    for _ in range(1, len(self.tabu)):
        i = self.tabu[_ - 1]
        j = self.tabu[_]
        if self.colony.update_strategy == 1:  # ant-quality system
            self.pheromone_delta[i][j] = self.colony.Q
        elif self.colony.update_strategy == 2:  # ant-density system
            # noinspection PyTypeChecker
            self.pheromone_delta[i][j] = self.colony.Q / self.graph.matrix[i][j]
        else:  # ant-cycle system
            self.pheromone_delta[i][j] = self.colony.Q / self.total_cost
```

```python
def _update_pheromone(self, graph: Graph, ants: list):
    for i, row in enumerate(graph.pheromone):
        for j, col in enumerate(row):
            graph.pheromone[i][j] *= self.rho
            for ant in ants:
                graph.pheromone[i][j] += ant.pheromone_delta[i][j]
```

$$\tau_{ij}(t+1) = \rho\tau_{ij}(t) + \Delta\tau_{ij}$$

Tarea

- Ejecutar el código ejemplo de implementación de ACO explicado para resolver el problema del viajero con las 45 ciudades dadas en el archivo *coord.txt*
- Comparar tiempo de ejecución, solución encontrada, convergencia de los tres métodos.